Hackathon IA pour tous : Roadmap tech personnalisée

Équipe de 3 personnes, durée : 3 heures, livrable : un prototype fonctionnel

1. Énoncé du projet

Thème : l'IA pour tous.

Problème : les roadmaps existantes (comme roadmap.sh) sont génériques : le même parcours pour tout le monde. Or un apprenant africain n'a pas le même temps, le même budget, la même connexion ni les mêmes acquis qu'un autre. Beaucoup se perdent entre trop de ressources, souvent payantes ou en anglais seulement.

Solution : une application web où l'utilisateur répond à quelques questions, et l'IA génère une **roadmap sur mesure** :

- des étapes ordonnées, avec une durée estimée pour chacune ;
- des ressources **gratuites** choisies dans une base vérifiée ;
- un mini-projet par étape ;
- un jalon final (par exemple « prêt à postuler »).

La roadmap saute ce que l'utilisateur sait déjà, s'adapte à son temps disponible et à ses contraintes, et s'exporte en texte léger lisible hors ligne.

Questions posées à l'utilisateur :

- Objectif (backend, frontend, mobile, data, DevOps…)
- Niveau et compétences déjà acquises
- Temps disponible par semaine
- Contraintes (connexion faible, ordinateur peu puissant, budget zéro)
- Langue préférée

Public cible : étudiants, autodidactes et reconversions, y compris les développeurs qui veulent changer de spécialité.

MVP (obligatoire) : formulaire → génération de la roadmap → affichage par étapes → export.

Hors périmètre : comptes utilisateurs, base de données, recherche web en direct, graphique de roadmap complexe.

2. Stack technique

Couche	Choix	Pourquoi
Langage	Python 3.10+	Simple et rapide à écrire
Interface	Streamlit	Formulaire et affichage sans HTML/CSS
IA	Groq API (Llama 3.3 70B)	Rapide, offre gratuite
Format de sortie	JSON structuré	Permet de dessiner la roadmap proprement
Base de ressources	Fichier `ressources.json` (environ 20 à 30 entrées)	Liens vérifiés, aucune URL inventée
Visuel	Cartes par étape, ou diagramme Mermaid en bonus	Rendu type roadmap
Export	Markdown (PDF en bonus)	Léger, lisible hors ligne
Versionnage	Git + GitHub	Travail à 3 sans conflits
Déploiement	Streamlit Community Cloud	Un lien à montrer au jury

Architecture : Formulaire → Prompt + base de ressources → LLM → JSON → Affichage par étapes → Export

Structure du dépôt : `app.py`, `generation.py`, `prompts.py`, `ressources.json`, `requirements.txt`, `.gitignore` (avec `.env`).

Exemple de sortie JSON attendue :

```
{
  "objectif": "Développeur backend",
  "duree_totale_semaines": 20,
  "etapes": [
    {
      "titre": "Bases de Python",
      "duree_semaines": 3,
      "competences": ["variables", "fonctions", "POO"],
      "ressources": ["id_ressource_1", "id_ressource_2"],
      "mini_projet": "Gestionnaire de dépenses en ligne de commande"
    }
  ]
}
```

L'IA ne renvoie que des identifiants de ressources présents dans `ressources.json`. L'application retrouve ensuite le nom et le lien réels, ce qui évite les liens inventés.

3. Contrat d'interface (à valider dans les 15 premières minutes)

- `generer_roadmap(profil: dict) -> dict` : prend les réponses du formulaire, renvoie le JSON de la roadmap (Personne A)
- `charger_ressources() -> dict` : charge la base de ressources vérifiées (Personne A)
- `exporter_markdown(roadmap: dict) -> str` : transforme la roadmap en texte téléchargeable (Personne C)

La Personne B développe l'interface avec une roadmap d'exemple en dur, puis branche la vraie fonction.

4. Répartition des tâches

Personne	Rôle	Responsabilités	Fichiers
A	IA et données	Base de ressources vérifiées, prompt, sortie JSON valide, gestion des erreurs de format	`generation.py`, `prompts.py`, `ressources.json`

Personne	Rôle	Responsabilités	Fichiers
B	Interface	Formulaire, affichage des étapes en cartes, suivi de progression (cases à cocher)	`app.py`
C	Export, déploiement, démo	Export Markdown/PDF, GitHub, déploiement, slides et pitch	`exporter_markdown()`, `requirements.txt`

5. Étapes de développement (cycle de vie adapté à 3 heures)

1. **Analyse du besoin (10 min) :** rédiger en trois phrases le qui, le problème et la solution. Fixer le MVP.
2. **Conception (20 min) :** dessiner l'écran du formulaire et l'écran de la roadmap, valider le format JSON et le contrat d'interface.
3. **Environnement (15 min) :** créer le dépôt Git, installer les dépendances, récupérer la clé API Groq, la mettre dans `.env`.
4. **Développement (1h15) :** chacun travaille sur son module en parallèle (voir le planning ci-dessous).
5. **Intégration (30 min) :** brancher les modules ensemble, tester de bout en bout.
6. **Tests (20 min) :** tester 3 profils différents, vérifier que le JSON est toujours valide, corriger.
7. **Déploiement (15 min) :** pousser sur GitHub, déployer sur Streamlit Cloud avec la clé dans les Secrets.
8. **Présentation (15 min) :** préparer et répéter le pitch.

6. Planning heure par heure

Horaire	Phase	A	B	C
0h00 à 0h30	Cadrage	Clé API, premier appel LLM	Structure Streamlit vide	Dépôt GitHub, `.gitignore`, structure du projet

Horaire	Phase	A	B	C
0h30 à 1h30	Développement parallèle	Base de 20 à 30 ressources, prompt, sortie JSON	Formulaire et affichage avec roadmap d'exemple	Export Markdown, `requirements.txt`
1h30	Synchro (10 min)	Chacun montre son livrable		
1h30 à 2h30	Intégration	Tests sur 3 profils, correction du prompt, gestion des JSON invalides	Branchement de la vraie fonction, spinner, messages d'erreur, cases à cocher	Déploiement, export dans l'interface
2h30	Gel du code	Test complet de bout en bout, tous ensemble		
2h30 à 3h00	Finition et démo	Préparer 2 profils de démo	Texte du pitch (problème et impact)	Slides (3 max), vidéo de secours, répétition

7. Prompt de départ (pour la Personne A)

```
Tu es un conseiller en carrière tech pour des apprenants africains.
Crée une roadmap personnalisée à partir du profil fourni.
Règles :
- Réponds uniquement en JSON valide, sans texte autour, selon le schéma donné.
- Choisis les ressources uniquement parmi la liste fournie (par identifiant).
- Saute les compétences déjà acquises.
- Adapte la durée de chaque étape au temps disponible par semaine.
- Privilégie les ressources gratuites et légères si la connexion est faible.
- Chaque étape a un mini-projet concret.
```

8. Plan de présentation (2 à 3 minutes)

Qui	Contenu	Durée
B	Le problème : roadmaps génériques, ressources hors de portée	30 s
C	Démo en direct : deux profils très différents donnent deux roadmaps différentes	1 min 30
A	Comment ça marche : base vérifiée, JSON, pas de liens inventés	30 s
Tous	Feuille de route : langues africaines, suivi de progression, communauté qui enrichit la base	20 s

Astuce de démo : montrez deux profils opposés (par exemple un débutant avec 5 h par semaine et connexion faible, puis un développeur confirmé avec 20 h par semaine). Voir deux roadmaps différentes prouve la personnalisation.

9. Règles d'équipe

1. Une branche Git par personne, fusion dans `main` aux points de synchro.
2. Ne pas modifier le fichier d'un autre sans le prévenir.
3. Ne jamais mettre la clé API dans le code ni sur GitHub.
4. Si quelqu'un est bloqué plus de 15 minutes, il le dit immédiatement.
5. Gel du code à 2h30 : seulement des corrections de bugs ensuite.

10. Risques et priorités

Risques principaux :

- **JSON invalide** renvoyé par le LLM : prévoir un `try/except` et une nouvelle tentative automatique.
- **Liens inventés** : résolus par la base de ressources vérifiées.
- **Wifi du hackathon** : garder la version locale et une vidéo de secours.

À sacrifier dans cet ordre si le temps manque : (1) le diagramme Mermaid, (2) l'export PDF, (3) le suivi de progression par cases à cocher, (4) le déploiement web.

À ne jamais sacrifier : formulaire → roadmap personnalisée → affichage par étapes.

11. Feuille de route (après le hackathon)

- Base de ressources enrichie par la communauté (programmes, bourses, formations locales).
- Sorties en langues africaines.
- Suivi de progression avec comptes utilisateurs.
- Lien avec un outil de résumé de vidéos pour transformer chaque étape en cours.